Smart Expense & Budget Management Application

Software Requirements Specification (SRS)

Version	Version 1.1
Date	September 8, 2026
Course / Department	Department of CSE – UE23CS341A

1. Introduction

1.1 Purpose

This document specifies the requirements for the Smart Expense & Budget Management Application (SmartBudget), a web-based personal-finance application implemented as a React frontend with a Node.js/Express backend and MongoDB persistence. It defines the currently implemented scope, user interactions, external interfaces, functional requirements, quality requirements, constraints, assumptions, and data entities. The document is intended for developers, project evaluators, testers, and stakeholders.

1.2 Scope

SmartBudget allows an individual user to create an account, sign in, maintain a monthly income and overall monthly budget target, record financial transactions, import transactions from a CSV bank statement, view spending analytics, export transaction reports as CSV, receive in-app budget/financial alerts, and manage savings goals. The application also contains a simulated automatic-expense-detection demonstration. Bank-account synchronization, real-time bank/UPI webhooks, email delivery, administrator functions, custom category CRUD, per-category budget limits, transaction editing, password reset, and PDF report export are not part of the current implemented release.

1.3 Document Conventions

The word “shall” denotes a mandatory requirement. “Should” denotes a recommended quality or deployment condition. Functional requirements are identified as FR-x.x; non-functional requirements are identified as NFR-x.x; user requirements are identified as UR-x.

1.4 User Requirements

UR-1. The user shall be able to create an account using name, email, and password.
UR-2. The user shall be able to sign in and sign out.
UR-3. The user shall be able to set monthly income and an overall monthly budget target.
UR-4. The user shall be able to add, view, and delete income/expense transactions.
UR-5. The user shall be able to upload a CSV bank statement and have supported rows parsed and automatically categorized.
UR-6. The user shall be able to view dashboards, analytics, budget utilization, and personalized in-app insights.
UR-7. The user shall be able to create, view, and delete savings goals.
UR-8. The user shall be able to export transaction/report data as CSV.

1.5 Domain Requirements

DR-1. Each stored transaction shall be associated with a user.
DR-2. Transaction amounts shall represent income as positive values and expenses as negative values in the application data model.
DR-3. Budget utilization shall be calculated from expense transactions against the user's overall monthly budget target.
DR-4. Financial data belonging to one user shall not be intentionally exposed in another user's normal application session.
DR-5. Password credentials shall not be stored or returned in plaintext.

1.6 Definitions, Acronyms, and Abbreviations

SRS — Software Requirements Specification
UI — User Interface
API — Application Programming Interface

REST — Representational State Transfer
JSON — JavaScript Object Notation
CSV — Comma-Separated Values
JWT — JSON Web Token; reserved terminology for token-based authentication, although the current implementation does not issue JWTs
MongoDB — Document-oriented database used by the current backend
Mongoose — ODM library used by the Node.js backend
Budget — User-defined overall monthly spending target
Savings Goal — User-defined financial target with target amount, current amount, and optional deadline
Statement Import — Client-side parsing of a CSV file containing transaction rows
Auto-categorization — Keyword-based assignment of a transaction to a predefined category

1.7 References

1. IEEE Std. 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
2. React documentation and project source code.
3. Node.js/Express project source code.
4. Mongoose/MongoDB project source code.
5. Project README and sample bank statement CSV supplied with the SmartBudget implementation.

2. Overall Description

2.1 Product Perspective

SmartBudget is a standalone client-server web application. The React/Vite frontend communicates with an Express API. The backend uses Mongoose to persist users, transactions, and savings goals in MongoDB. The current prototype is configured to use localhost port 5001 for the backend API and port 5173 for the Vite frontend during development. The frontend also performs CSV parsing, automatic categorization, chart rendering, and several notification/insight calculations locally.

2.2 Product Functions

• Account registration and login.
• Monthly income and overall monthly budget settings.
• Manual transaction creation and deletion.
• Transaction history display with income/expense views.
• CSV statement upload using Papa Parse.
• Keyword-based automatic categorization of supported statement narrations.
• Local removal of imported CSV transactions.
• Simulated automatic expense detection for demonstration.
• Dashboard with income, expenses, balance, spending overview, category breakdown, recent transactions, alerts, and insights.
• Budget utilization display based on the overall monthly budget.
• Analytics using charts for income versus expenses and category spending.
• CSV report generation/download.
• Savings-goal creation, viewing, and deletion.
• Light/dark theme selection persisted in browser local storage.

2.3 User Classes

Individual User — primary application user; basic familiarity with web applications is sufficient.
Guest/Visitor — can access the authentication/landing interface before signing in.
Administrator — not implemented in the current release; administrative functionality described in the previous SRS

has therefore been removed from the current requirements.

2.4 Operating Environment

Client: modern desktop, tablet, or mobile browser capable of running React/Vite frontend assets.
Frontend development server: Vite, normally on localhost:5173.
Backend development server: Node.js/Express, currently on localhost:5001.
Database: MongoDB, local by default at mongodb://127.0.0.1:27017/smartbudget or a MongoDB Atlas URI supplied through MONGODB_URI.
Runtime packages include React, React DOM, Recharts, Papa Parse, Express, Mongoose, MongoDB, bcryptjs, and CORS.

2.5 Constraints

• The current implementation uses JavaScript/React for the frontend and Node.js/Express for the backend.
• MongoDB with Mongoose is the implemented persistence layer.
• The backend must be able to connect to MongoDB before serving normal data operations.
• The prototype uses a development HTTP connection to localhost; production deployment should use HTTPS/TLS.
• CSV statement import is client-side and is not persisted to the backend.
• The automatic-expense-detection card is simulated and does not represent a live bank integration.

2.6 Assumptions and Dependencies

• Users have access to a supported web browser.
• A MongoDB instance is available to the backend.
• Required npm dependencies are installed.
• Imported statement files use supported CSV columns such as Date, Narration/Description, Debit, and Credit.
• Third-party email/SMS delivery is not required by the current implementation.
• The project may evolve to add authentication tokens, bank integrations, richer budgets, and additional export formats in a future release.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface

The UI shall provide navigation for Dashboard, Transactions, Budgets, Analytics, Reports, and Savings Goals, plus account settings and authentication screens. It shall provide forms for adding transactions, saving monthly settings, creating savings goals, and uploading CSV statements. Charts shall be used for category and time-based analytics. An in-app notification area shall show budget/financial alerts when applicable.

3.1.2 Hardware Interface

No specialized hardware is required. The application shall run on standard computing devices with a supported web browser. A server or development machine capable of running Node.js and connecting to MongoDB is required for the backend.

3.1.3 Software Interfaces

Frontend: React with Vite.
Backend: Node.js with Express.
Database: MongoDB accessed through Mongoose.

Authentication: email/password with bcryptjs password hashing; the current implementation does not issue JWT tokens.
Charts: Recharts.
CSV parsing/report generation: Papa Parse and browser Blob/download APIs.
Icons/UI assets: lucide-react.

3.1.4 Communication Interfaces

The frontend communicates with the backend through HTTP REST-style JSON requests during local development. The current API base is http://localhost:5001/api. A production deployment should use HTTPS/TLS and an appropriate deployed API origin.

3.2 Functional Requirements

The following requirements describe the behaviour of the current implementation and deliberately exclude features that are only proposed or described in the previous SRS.

3.3 Non-Functional Requirements

3.3 Non-Functional Requirements

Quality requirements for the current release are listed below.

3.4 Data Requirements

3.4 Data Requirements

Entities in the current backend are User, Transaction, and Goal. The fields below reflect the Mongoose models used by the implementation.

3.5 API Requirements

The current backend exposes the following principal API operations:
POST /api/signup — create a user.
POST /api/login — authenticate a user.
PUT /api/users/:id/settings — update monthly income and monthly budget.
GET /api/users — list registered users (development/demo utility; should be restricted or removed in production).
POST /api/transactions — create a transaction.
GET /api/transactions/:userId — retrieve a user's transactions.
DELETE /api/transactions/:id — delete a transaction.
POST /api/goals — create a savings goal.
GET /api/goals/:userId — retrieve a user's savings goals.
DELETE /api/goals/:id — delete a savings goal.

3.6 Design Constraints

The frontend uses React/Vite and the backend uses Node.js/Express. MongoDB is the persistence technology in the current implementation. The local development backend listens on port 5001. Any production deployment should replace development localhost configuration, restrict CORS appropriately, protect administrative/development endpoints, and use HTTPS/TLS.

4. Traceability

4.1 User-to-Functional Requirement Mapping

UR-1 → FR-1.1–FR-1.5
UR-2 → FR-1.6–FR-1.7
UR-3 → FR-2.1–FR-2.3
UR-4 → FR-3.1–FR-3.6
UR-5 → FR-4.1–FR-4.7
UR-6 → FR-6.1–FR-7.5
UR-7 → FR-9.1–FR-9.4
UR-8 → FR-8.1–FR-8.3

4.2 Out-of-Scope / Future Enhancements

The following features were intentionally removed from the current requirements because they are not implemented in the supplied project: password reset by email, editable transactions, user-defined category CRUD, per-category budget limits, email/SMS alert delivery, administrator account management, PDF report generation, direct bank-account integration, live bank/UPI webhook ingestion, multi-currency support, and shared household budgeting. They may be specified in a future SRS revision if implemented.

5. Acceptance Criteria

5.1 Core Acceptance Criteria

The release shall be considered functionally aligned with this SRS when a tester can: (1) register a new account with a valid password; (2) log in with the created credentials; (3) save monthly income and overall budget values; (4) add and delete income/expense transactions; (5) upload the supplied sample CSV and see parsed/categorized transactions; (6) see dashboard, budget, analytics, and insight values update from available transactions; (7) create and delete a savings goal; and (8) download a CSV report. The tester shall not expect the explicitly out-of-scope features listed in Section 4.2.

6. Appendices

Appendix A Current Technology Stack

Frontend: React 18 + Vite
Backend: Node.js + Express
Database: MongoDB + Mongoose
Authentication/Password Security: bcryptjs
Charts: Recharts
CSV Parsing: Papa Parse
Icons: lucide-react
Development API: localhost:5001
Development Frontend: localhost:5173

Appendix B Current Categories

Food, Transport, Shopping, Entertainment, Income, and Other are represented by the current categorization logic. Imported statement narrations are classified by keyword rules; unsupported narrations are assigned Other. The current implementation does not provide a backend-managed custom-category table.

3.2 Functional Requirements — Detailed Tables

3.2.1 Account Management

ID	Requirement
FR-1.1	The system shall allow a new user to register using name, email address, and password.
FR-1.2	The system shall reject registration when required fields are missing.
FR-1.3	The system shall require a password of at least 6 characters.
FR-1.4	The system shall prevent creation of another account with an existing email address.
FR-1.5	The system shall hash passwords using bcryptjs before persistence.
FR-1.6	The system shall allow a registered user to log in using email and password.
FR-1.7	The system shall allow the user to log out of the current application session.

3.2.2 Financial Settings

ID	Requirement
FR-2.1	The system shall allow the user to set a non-negative monthly income value.
FR-2.2	The system shall allow the user to set a non-negative overall monthly budget value.
FR-2.3	The system shall persist monthly income and monthly budget settings for the user.

3.2.3 Transaction Management

ID	Requirement
FR-3.1	The system shall allow the user to add a transaction with name, category, amount, date, optional payment method, and optional notes.
FR-3.2	The system shall represent income amounts as positive values and expense amounts as negative values.
FR-3.3	The system shall allow the user to view transactions associated with that user.
FR-3.4	The system shall allow the user to delete an existing transaction.
FR-3.5	The system shall validate that a manually entered transaction contains required fields and a numeric amount.
FR-3.6	The system shall support viewing all transactions and separate income/expense views in the UI.

3.2.4 Statement Import & Categorization

ID	Requirement
FR-4.1	The system shall allow the user to select a CSV statement file from the UI.
FR-4.2	The system shall parse supported CSV rows using headers including Date, Narration/Description, Debit, and Credit.
FR-4.3	The system shall ignore rows without a narration/description or without a non-zero debit/credit amount.
FR-4.4	The system shall automatically categorize supported narrations using predefined keyword rules and assign Other when no rule matches.
FR-4.5	The system shall add successfully parsed statement transactions to the current client-side transaction list.
FR-4.6	The system shall allow the user to remove imported statement transactions from the current client-side list.

ID	Requirement
FR-4.7	The system shall clearly indicate import success, failure, or absence of usable rows.

3.2.5 Automatic Detection Demonstration

ID	Requirement
FR-5.1	The system shall provide a user-triggered automatic-expense-detection demonstration.
FR-5.2	The demonstration shall add a simulated transaction from a predefined incoming transaction feed.
FR-5.3	The system shall not represent the simulated feed as a live bank, UPI, or payment-gateway integration.

3.2.6 Budget Monitoring & Alerts

ID	Requirement
FR-6.1	The system shall calculate total expenses from negative transaction amounts.
FR-6.2	The system shall calculate budget utilization as total expenses divided by the user's overall monthly budget.
FR-6.3	The system shall show an alert when spending reaches or exceeds 80% of the overall monthly budget.
FR-6.4	The system shall show an over-budget alert when spending exceeds 100% of the overall monthly budget.
FR-6.5	The system shall display budget progress in the dashboard and Budgets screen.

3.2.7 Analytics & Insights

ID	Requirement
FR-7.1	The system shall display total income, total expenses, and balance derived from available transaction data.
FR-7.2	The system shall display expense totals by category.
FR-7.3	The system shall display income-versus-expense trends over transaction dates/months.
FR-7.4	The system shall display top spending categories.
FR-7.5	The system shall display contextual in-app insights based on spending and budget data.

3.2.8 Reports

ID	Requirement
FR-8.1	The system shall provide a Reports screen summarizing income, expenses, and budget performance.
FR-8.2	The system shall allow the user to download transaction/report data as CSV.
FR-8.3	The system shall generate the CSV report in the browser without requiring a server-side report-generation service.

3.2.9 Savings Goals

ID	Requirement
FR-9.1	The system shall allow the user to create a savings goal with name, target amount, current amount, and optional deadline.
FR-9.2	The system shall persist savings goals in MongoDB.

ID	Requirement
FR-9.3	The system shall display the user's saved savings goals.
FR-9.4	The system shall allow the user to delete a savings goal.

3.2.10 Preferences

ID	Requirement
FR-10.1	The system shall provide light and dark display themes.
FR-10.2	The system shall persist the selected theme in browser local storage.

3.3 Non-Functional Requirements — Detailed Table

ID	Quality Attribute	Requirement
NFR-1.1	**Security**	Passwords shall be stored using bcryptjs hashing and shall not be returned to the frontend as password hashes.
NFR-1.2	**Security**	Production client-server communication should use HTTPS/TLS.
NFR-1.3	**Security**	The application shall use user identifiers to associate transactions and savings goals with the corresponding account.
NFR-2.1	**Usability**	The UI shall provide clear labels, navigation, validation feedback, and empty-state messages for major screens.
NFR-2.2	**Usability**	The interface shall be responsive for desktop, tablet, and mobile-sized screens.
NFR-3.1	**Performance**	Normal local development requests should complete without unnecessary blocking work; large statement files should be handled with clear import feedback.
NFR-3.2	**Performance**	Charts and summaries shall be derived from the available transaction data rather than fixed historical demo values.
NFR-4.1	**Reliability**	The backend shall return an appropriate error response when MongoDB is unavailable or a requested record does not exist.
NFR-4.2	**Reliability**	Client-side import failures shall present an understandable error message rather than silently failing.
NFR-5.1	**Maintainability**	Frontend screens shall be organized into reusable React components.
NFR-5.2	**Maintainability**	Backend data entities shall be represented by separate Mongoose models.
NFR-6.1	**Portability**	The application shall operate in current versions of major browsers that support the required React, JavaScript, File API, and local-storage features.

3.4 Data Requirements — Entity Definitions

Entity	Fields	Purpose
User	name, email, password_hash, monthly_income, monthly_budget, created_at	Stores account credentials and monthly financial targets.
Transaction	user_id, name, category, amount, txn_date, payment_method, notes, created_at	Stores income/expense entries; amount sign distinguishes income and expense.
Goal	user_id, name, target, current, deadline, created_at	Stores savings targets and progress values.

3.5 API Requirements — Endpoint Summary

Method	Endpoint	Purpose
POST	/api/signup	Create user account
POST	/api/login	Authenticate user
PUT	/api/users/:id/settings	Save monthly income and budget
GET	/api/users	List users; development/demo utility
POST	/api/transactions	Create transaction
GET	/api/transactions/:userId	Get user's transactions
DELETE	/api/transactions/:id	Delete transaction
POST	/api/goals	Create savings goal
GET	/api/goals/:userId	Get user's savings goals
DELETE	/api/goals/:id	Delete savings goal

Traceability Matrix

User Req.	Mapped FRs	Coverage
UR-1	FR-1.1–FR-1.5	Account creation and password handling
UR-2	FR-1.6–FR-1.7	Login and logout
UR-3	FR-2.1–FR-2.3	Monthly income and overall budget
UR-4	FR-3.1–FR-3.6	Transaction entry, viewing and deletion
UR-5	FR-4.1–FR-4.7	CSV statement import and categorization
UR-6	FR-6.1–FR-7.5	Budget monitoring, analytics and insights
UR-7	FR-9.1–FR-9.4	Savings goals
UR-8	FR-8.1–FR-8.3	CSV reports